tf.tensor_scatter_nd_sub Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/tensor_scatter_nd_sub

> *Subtracts sparse updates from an existing tensor according to indices.*

Function Signatures

```
tf.tensor_scatter_nd_sub( tensor: Annotated[Any,
TV_TensorScatterSub_T], indices: Annotated[Any,
TV_TensorScatterSub_Tindices], updates: Annotated[Any,
TV_TensorScatterSub_T], name=None ) -> Annotated[Any,
TV_TensorScatterSub_T]
```

Code Examples

```
tf.tensor_scatter_nd_sub(
tensor: Annotated[Any, TV_TensorScatterSub_T],
indices: Annotated[Any, TV_TensorScatterSub_Tindices],
updates: Annotated[Any, TV_TensorScatterSub_T],
name=None
) -> Annotated[Any, TV_TensorScatterSub_T]
```

```
tf.tensor_scatter_nd_sub(
tensor: Annotated[Any, TV_TensorScatterSub_T],
indices: Annotated[Any, TV_TensorScatterSub_Tindices],
updates: Annotated[Any, TV_TensorScatterSub_T],
name=None
) -> Annotated[Any, TV_TensorScatterSub_T]
```

```
indices.shape[-1] <= shape.rank
```


```
indices.shape[-1] <= shape.rank
```


```
indices.shape[:-1] + shape[indices.shape[-1]:]
```


```
indices.shape[:-1] + shape[indices.shape[-1]:]
```


```
indices = tf.constant([[4], [3], [1], [7]])
updates = tf.constant([9, 10, 11, 12])
tensor = tf.ones([8], dtype=tf.int32)
updated = tf.tensor_scatter_nd_sub(tensor, indices, updates)
print(updated)
```


```
indices = tf.constant([[4], [3], [1], [7]])
updates = tf.constant([9, 10, 11, 12])
tensor = tf.ones([8], dtype=tf.int32)
updated = tf.tensor_scatter_nd_sub(tensor, indices, updates)
print(updated)
```


Documentation

Subtracts sparse updates from an existing tensor according to indices.

View aliases

Compat aliases for migration

See
Migration guide for
more details.

tf.compat.v1.tensor_scatter_nd_sub, tf.compat.v1.tensor_scatter_sub

Used in the notebooks

• Introduction to tensor slicing

This operation creates a new tensor by subtracting sparse updates from the passed in tensor.
This operation is very similar to tf.scatter_nd_sub, except that the updates are subtracted from an existing tensor (as opposed to a variable). If the memory for the existing tensor cannot be re-used, a copy is made and updated.

indices is an integer tensor containing indices into a new tensor of shape shape. The last dimension of indices can be at most the rank of shape:

The last dimension of indices corresponds to indices into elements
(if indices.shape[-1] = shape.rank) or slices
(if indices.shape[-1] < shape.rank) along dimension indices.shape[-1] of shape. updates is a tensor with shape

The simplest form of tensor_scatter_sub is to subtract individual elements from a tensor by index. For example, say we want to insert 4 scattered elements in a rank-1 tensor with 8 elements.

In Python, this scatter subtract operation would look like this:

The resulting tensor would look like this:

We can also, insert entire slices of a higher rank tensor all at once. For example, if we wanted to insert two slices in the first dimension of a rank-3 tensor with two matrices of new values.

In Python, this scatter add operation would look like this:

The resulting tensor would look like this:

Note that on CPU, if an out of bound index is found, an error is returned. On GPU, if an out of bound index is found, the index is ignored.

Returns